

Algorithmique & Programmation

Introduction à la programmation pour le TAL

Eric Jordan

07 novembre 2025

1 Les Listes

2 Exercices

1 Les Listes

2 Exercices

Définition d'une liste

- ▶ Une **liste** est une collection ordonnée et modifiable d'éléments.
- ▶ Définie avec des crochets : `[ ]`.

Exemples

- ▶ `nombres = [1, 2, 3, 4]`
- ▶ `mélange = [1, "a", 3.5, True]`
- ▶ `vide = []`

Slicing, accès, len

- ▶ Accès par indice : `ma_liste[0]`, `ma_liste[-1]`
- ▶ Portion (slicing) : `ma_liste[1:4]`, `ma_liste[:3]`, `ma_liste[2:]`
- ▶ Taille : `len(ma_liste)`

Exemple

```
l = [10, 20, 30, 40, 50]
l[1:4] → [20, 30, 40]
len(l) → 5
```

Appartenance : in

- ▶ Vérifie si un élément est présent dans la liste.
- ▶ `x in ma_liste` retourne `True` ou `False`.

Exemple

```
3 in [1, 2, 3, 4] → True
```

```
"a" in [1, 2, 3] → False
```

Opérateurs : *, +

- ▶ + concatène deux listes.
- ▶ * répète une liste.

Exemples

$[1, 2] + [3, 4] \rightarrow [1, 2, 3, 4]$

$[0] * 5 \rightarrow [0, 0, 0, 0, 0]$

Modification, remplacement, suppression

- ▶ Remplacement : `ma_liste[i] = nouvelle_valeur`
- ▶ Suppression : `del ma_liste[i]` ou `ma_liste.remove(x)`

Exemple

```
l = [1, 2, 3, 4]
l[1] = 99 → [1, 99, 3, 4]
del l[2] → [1, 99, 4]
```


Méthodes

- ▶ `l.append(x)` : ajoute `x` à la fin
- ▶ `l.extend(L2)` : concatène une autre liste, `L2`
- ▶ `l.insert(i, x)` : insère `x` à la position `i` (sans supprimer la valeur déjà présente)
- ▶ `l.index(x)` : donne la première position de `x`
- ▶ `l.sort()` : trie la liste, **en la modifiant** (in-place)
- ▶ `l.reverse()` : inverse l'ordre

Exemple

```
l = [3,1,2]; l.sort() → [1,2,3]
l.append(4) → [1,2,3,4]
```

Modification via slicing

- ▶ On peut remplacer une portion de liste par une autre.
- ▶ Possible de changer la taille.

Exemple

```
l = [1, 2, 3, 4, 5]
```

```
l[1:3] = [9, 9, 9] → [1, 9, 9, 9, 4, 5]
```

```
l[:2] = [0, 0, 0] → [0, 9, 0, 9, 0, 5]
```

Conversion avec list()

- ▶ La fonction `list()` convertit un **itérable** en liste.
- ▶ Alternative à `.copy()` (quand on veut faire une copie sans modifier la liste originale)

Exemples

```
list("abc") → ['a', 'b', 'c']
```

```
list(range(3)) → [0, 1, 2]
```

Parcours avec for

- ▶ On peut parcourir directement les éléments d'une liste.

Exemple

```
for x in [1, 2, 3]:  
    print(x)
```

Résultat

```
1  
2  
3
```

Contrôle de boucle : break, continue, pass

- ▶ break : interrompt la boucle immédiatement.
- ▶ continue : passe à l'itération suivante.
- ▶ pass : ne fait rien (instruction vide utilisée comme "bouchon").

Exemple

```
for x in [1, 2, 3, 4, 5]:  
    if x == 3: continue  
    if x == 5: break  
    print(x)  
  
# pass peut être utilisé comme placeholder :  
for x in range(3): pass
```

Résultat

```
1  
2  
4
```

Functions

- ▶ `sorted(liste)` : retourne une nouvelle liste triée.
- ▶ `enumerate(liste)` : retourne couples (indice, valeur).

Exemples

```
sorted([3,1,2]) → [1,2,3]
```

```
list(enumerate(['a','b'])) → [(0,'a'), (1,'b')]
```

Parcours avec enumerate

- ▶ Permet d'avoir à la fois l'indice et la valeur.

Exemple

```
fruits = ['pomme', 'banane', 'kiwi']  
for i, f in enumerate(fruits):  
    print(i, f)
```

Résultat

```
0 pomme  
1 banane  
2 kiwi
```

Listes compactes (list comprehensions)

- Syntaxe courte pour créer une liste à partir d'un itérable.

Syntaxe générale

```
[expression for élément in itérable (if condition)]
```

Exemples

```
[x**2 for x in range(5)] → [0,1,4,9,16]
```

```
[c.upper() for c in 'abc'] → ['A','B','C']
```


Listes compactes avec conditionnel

- ▶ On peut filtrer les éléments avec une condition.

Exemples

```
[x for x in range(10) if x%2==0] → [0,2,4,6,8]
```

```
[x**2 for x in range(5) if x>2] → [9,16]
```

1 Les Listes

2 Exercices

Exercice 1 — Création et accès

Soit la liste : $L = [10, 20, 30, 40, 50]$

- 1 Afficher le premier et le dernier élément.
- 2 Extraire les trois éléments du milieu.
- 3 Créer une nouvelle liste contenant les éléments aux indices pairs.

Exercice 2 — Boucles et conditions

Soit la liste `nombres = [1, 4, 7, 12, 15, 18]`.

- 1 Parcourez la liste et affichez uniquement les nombres impairs.
- 2 Arrêtez la boucle quand vous rencontrez un nombre supérieur à 15.
- 3 Ignorez le nombre 7 sans interrompre la boucle.

Exercice 3 — Méthodes de liste

Soit `fruits = ['pomme', 'banane', 'kiwi']`.

- 1 Ajoutez 'orange' à la fin.
- 2 Insérez 'mangue' en deuxième position.
- 3 Supprimez 'banane'.
- 4 Triez la liste par ordre alphabétique.
- 5 Inversez l'ordre des éléments.

Exercice 4 — Listes compactes

- 1 Créez une liste des cubes des nombres de 0 à 10.
- 2 Créez une liste des nombres impairs entre 0 et 20.
- 3 Créez une liste contenant 'pair' ou 'impair' selon chaque nombre de 0 à 9.

Exercice 5 — Mise en pratique complète

Soit `notes = [12, 5, 8, 15, 10, 19, 7]`.

- 1 Affichez toutes les notes supérieures ou égales à 10.
- 2 Créez une nouvelle liste contenant uniquement ces notes.
- 3 Affichez la moyenne des notes valides.
- 4 Si la moyenne dépasse 12, affichez “Très bien”, sinon “Peut mieux faire”.

Exercices — Manipulations de listes I

Trouver l'indice d'un élément

Soit `nombres = [3, 5, 7, 2, 7, 9]`. Trouver l'indice de la première occurrence de 7. Bonus : afficher les indices de **tous** les 7.

Combiner deux listes

Soient `mois = ['janvier', 'février', 'mars']` et `jours = [31, 28, 31]`. Créez `['janvier', 31, 'février', 28, 'mars', 31]`. Utilisez une boucle ou une compréhension.

Exercices — Manipulations de listes II

Trouver le maximum

Soit `valeurs = [12, 45, 3, 67, 23, 9]`. Trouvez le plus grand élément avec une boucle.

Séparer selon une condition

Soit `nombres = [3, 4, 8, 5, 16, 7, 12]`. Créez deux listes : `div4` (divisibles par 4) et `autres` (le reste). Utilisez une boucle et une compréhension.

`eric.jordan@inalco.fr`